

CASA0029: Urban Data Visualisation

Using Web APIs for Real Time Data Viz

This practical uses Web Application Programming Interfaces (APIs) to request real time data for some simple visualisations. We will use APIs from Transport for London for real time London transit data, and an API from AQICN, an international network of air quality data.

Introducing Web APIs: Transport for London

At their most basic, Web APIs are a publicly accessible URLs (sometimes called endpoints) where data can be accessed, usually as a JSON file or XML file. Transport for London provide some very popular APIs that are the foundation of various journey planner software and apps in London. The following link provides a list of TfL APIs- <https://api.tfl.gov.uk/>

The TfL Unified API website allows you to explore the various APIs and see the sample data that is returned by each request. The parameters of the query are contained in the API URL. For example, to find out the service status of tube lines in London, you would use the following URL- <https://api.tfl.gov.uk/Line/Mode/tube/Status>

The screenshot displays the TfL Unified API interface. On the left is a sidebar with a list of API endpoints under various categories like AirQuality, BikePoint, Cabwise, Journey, Line, Mode, and Occupancy. The 'Line' category is selected, and the endpoint 'Gets the line status of for all lines for the given modes' is highlighted. The main panel shows the details for this endpoint, including the URL `GET /Line/Mode/(modes)/Status` and a 'Parameters' section with a 'modes' input field set to 'tube' and a 'detail' dropdown set to 'false'. Below this is a 'Test this endpoint' section with a 'TRY' button and a 'Response Type' dropdown set to 'application/json'. On the right, a 'RESPONSE SAMPLE' section shows a JSON object representing the API response, which includes metadata like 'id', 'name', 'modeName', and 'disruptions', as well as detailed information about the tube line, such as 'category', 'type', 'description', 'summary', 'additionalInfo', 'created', 'lastUpdate', 'affectedRoutes', 'lineId', 'lineName', 'direction', 'originName', 'destinationName', 'validTo', 'validFrom', 'routeSectionHaptanEntrySequence', 'stopPoint', 'platform', 'indicator', 'stopLetter', 'nodes', 'line', 'icCode', 'secCode', 'stopType', 'stationHaptan', 'accessibilitySummary', 'subHaptanCode', and 'lines'.

The URLs from the TfL site will work directly in your code. However, if you are setting up a website or app that is likely to receive considerable traffic, then you need to register your app with TfL. Then you can use the app id in your API requests- <https://api-portal.tfl.gov.uk/faq>

Creating a Real Time Tube Status Table

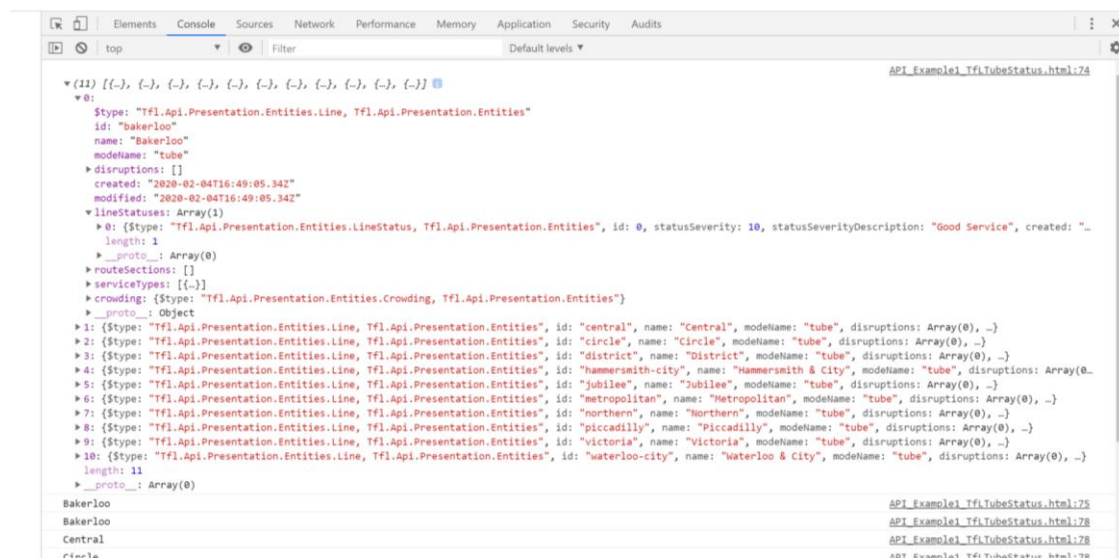
Our first example is a simple table that presents the live status of each tube line in London. This is the “*API_Example1_TfLTubeStatus.html*” file in the Moodle docs.

Line	Status
Bakerloo	Good Service
Central	Good Service
Circle	Good Service
District	Good Service
Hammersmith and City	Good Service
Jubilee	Good Service
Metropolitan	Good Service
Northern	Good Service
Piccadilly	Good Service
Victoria	Good Service
Waterloo and City	Good Service

There are some similarities here to the chart examples from Week 4 where we imported data from a CSV file using the `d3.csv()` method. This time we import data from a URL using the similar `d3.json()` method-

```
d3.json(https://api.tfl.gov.uk/line/mode/tube/status).then(function(StatusData) { ... })
```

This method imports JSON data from the requested URL, and then after it has been imported it runs the function. `StatusData` is a JSON object (or in this case an array of JSON objects) that contains the imported data. You can view the imported JSON data using `console.log(StatusData)`. You will be able to see that `StatusData` is quite a complicated hierarchy of JSON objects and arrays. This is a typical structure for data retrieved from APIs-



The information on the status of each line is the “`statusSeverityDescription`”, which is contained within two nested arrays. So for example, the Bakerloo line status is found at- `StatusData[0].lineStatuses[0].statusSeverityDescription`

The `API_Example1_TfLTubeStatus.html` example loops through the `StatusData` array, and writes the status of each line to the `TubeStatusTable`.

API_Example1_TfLTubeStatus.html

```
<!DOCTYPE html>
<html>
<head>
<title>Tf1 Tube Line Status Real Time Data Example</title>
<meta name="viewport" content="initial-scale=1.0, user-scalable=no" />

<script charset="utf-8" src="https://d3js.org/d3.v7.min.js"></script>

</head>

<body>

<!--Table that will present the current status of each tube line-->
<table id="TubeStatusTable">
  <tr>
    <th>Line</th>
    <th>Status</th>
  </tr>
  <tr>
    <td class="LineName" style='background-color: #B36305; color: #FFFFFF'>Bakerloo</td>
    <td class="LineStatus" id="bakerloo">Bakerloo</td>
  </tr>
  <tr>
    <td class="LineName" style='background-color: #E32017; color: #FFFFFF'>Central</td>
    <td class="LineStatus" id="central">Central</td>
  </tr>
  <tr>
    <td class="LineName" style='background-color: #FFD300; color: #000000'>Circle</td>
    <td class="LineStatus" id="circle">Circle</td>
  </tr>
  <tr>
    <td class="LineName" style='background-color: #00782A; color: #FFFFFF'>District</td>
    <td class="LineStatus" id="district">District</td>
  </tr>
  <tr>
    <td class="LineName" style='background-color: #F3A9BB; color: #000000'>Hammersmith and
City</td>
    <td class="LineStatus" id="hammersmith-city">Hammersmith and City</td>
  </tr>
  <tr>
    <td class="LineName" style='background-color: #A0A5A9; color: #000000'>Jubilee</td>
    <td class="LineStatus" id="jubilee">Jubilee</td>
  </tr>
  <tr>
    <td class="LineName" style='background-color: #9B0056; color: #FFFFFF'>Metropolitan</td>
    <td class="LineStatus" id="metropolitan">Metropolitan</td>
  </tr>
  <tr>
    <td class="LineName" style='background-color: #000000; color: #FFFFFF'>Northern</td>
    <td class="LineStatus" id="northern">Northern</td>
  </tr>
  <tr>
    <td class="LineName" style='background-color: #003688; color: #FFFFFF'>Piccadilly</td>
    <td class="LineStatus" id="piccadilly">Piccadilly</td>
  </tr>
  <tr>
    <td class="LineName" style='background-color: #0098D4; color: #FFFFFF'>Victoria</td>
    <td class="LineStatus" id="victoria">Victoria</td>
  </tr>
  <tr>
    <td class="LineName" style='background-color: #95CDBA; color: #000000'>Waterloo and
City</td>
    <td class="LineStatus" id="waterloo-city">Waterloo and City</td>
  </tr>
</table>
```

```

<script>
//This statement loads the JSON data for the TfL API, and then runs the function after the data is
loaded. StatusData is a JSON array
d3.json("https://api.tfl.gov.uk/line/mode/tube/status").then(function(StatusData) {

    console.log(StatusData); //Logs the StatusData JSON array in the console panel
    console.log(StatusData[0].name); //Logs the name of tube line that is first in the array,
    Bakerloo

    StatusData.forEach(function(line){
        console.log(line.name);
        var StatusDescription = (line.lineStatuses[0].statusSeverityDescription); //This is
where the line status is contained in the JSON data
        document.getElementById(line.id).innerHTML = StatusDescription; //Write the line
status to the corresponding cell in the HTML table
    });

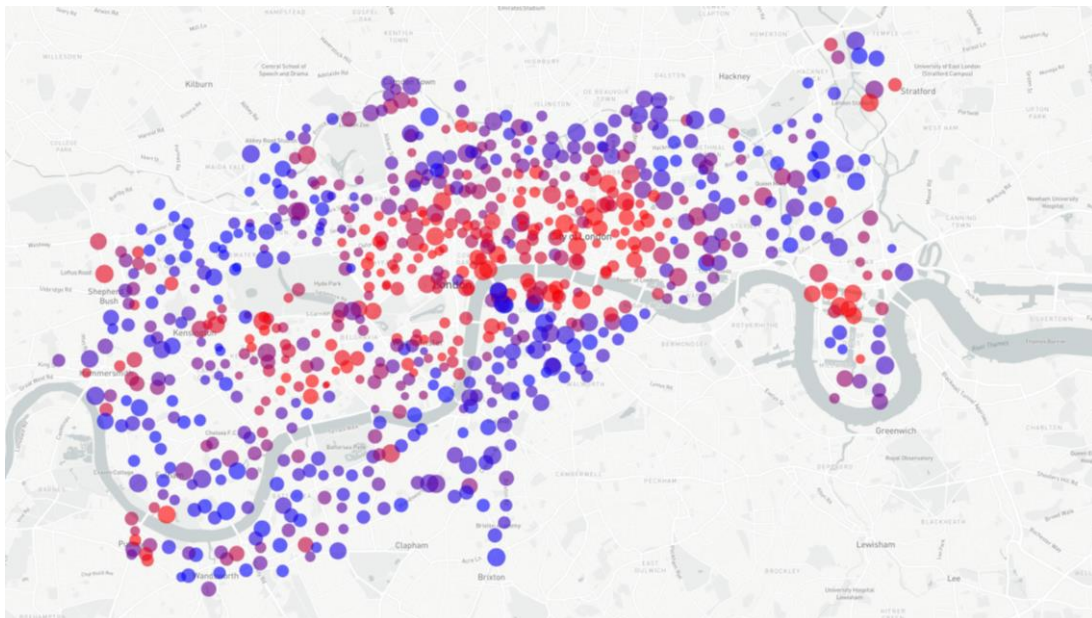
});

</script>
</body>
</html>

```

Mapping Data from APIs: TfL Bikeshare Data

As well as updating tables using live API data, we can also add real time spatial data to interactive maps. The most common format for exchanging spatial data online is the GeoJSON format. Some APIs provide GeoJSON data directly, other times you have to create a GeoJSON in your code. In the next example we will map data on TfL Bikeshare docking stations that comes from the TfL Bike Point API. We will use Mapbox GL to create a proportional circle map. Red stations are full of bikes, and blue stations are empty (similar to Ollie O'Brien's maps at <https://bikesharemap.com>)-



This example has many similarities to the Tube Entry Exits proportional circle example we created in Week 2 but this time we are going to get live data from the TfL API, rather than from our Tileset that we uploaded. So again we use the `d3.json()` method, this time using the Bike Point URL-

```
d3.json(https://api.tfl.gov.uk/bikepoint).then(function(BikeData) { ... })
```

This time the data is written to the JSON array BikeData. Once loaded, this is an array of 801 items, one for each docking station. The TfL API does not provide the option of returning GeoJSON data, so the example below creates a new GeoJSON and inserts the required data as a series of features, one feature for each docking station. This is done by looping through the BikeData array-

```
for (i = 0; i < BikeData.length; i++) { ... }
```

We extract the required data including the latitude, longitude, number of bikes, number of docks, and insert these into a “newFeature” variable. The “newFeature” is added to the GeoJSON features array using the ‘push’ method-

```
geojson.features.push(newFeature); //Add the new feature to the features array
```

API_Example2_TfLBikeShare.html

```
<!DOCTYPE html>
<html>
<head>
<title>TfL Bike Share Docking Stations Real Time Map</title>
<meta name="viewport" content="initial-scale=1.0, user-scalable=no" />

<link href="https://api.mapbox.com/mapbox-gl-js/v3.17.0/mapbox-gl.css" rel="stylesheet">
<script src="https://api.mapbox.com/mapbox-gl-js/v3.17.0/mapbox-gl.js"></script>

<script charset="utf-8" src="https://d3js.org/d3.v7.min.js"></script>

<style>

  body { margin: 0; padding: 0; }

  #title {
    position: absolute;
    background-color: #fff;
    box-shadow:0 1px 2px rgba(0, 0, 0, 0.20);
    border-radius: 3px;
    padding: 15px;
    margin-top: 15px;
    margin-left: 15px;
  }

  #title h2 {
    line-height: 24px;
    margin: 0;
  }

  #title p.credit {
    margin: 10px 0 0 0;
    padding: 0;
  }

  #map { position: absolute; top: 0; bottom: 0; width: 100%; };

</style>
</head>

<body>

  <div id="map"></div>

  <div id="title">
    <h2>London Bike Share Live Map</h2>
    <p class="credit">Data: <a href="https://api.tfl.gov.uk/">Transport for London API</a>.</p>
  </div>
```

```

<script>

mapboxgl.accessToken = 'pk.eyJ1IjoizHVuY2FuMjAwMSIsImEiOiIyRDhtOE1nIn0.OA5QmCprkPbOoxZog8HIow';

// Load a new map in the 'map' HTML div
var map = new mapboxgl.Map({
  container: 'map', // container id
  style: 'mapbox://styles/mapbox/light-v10', // Using the standard Mapbox Light style
  center: [-0.1, 51.5], // starting position [lng, lat]
  zoom: 12 // starting zoom
});

map.on('load', function() {

  //This statement loads the JSON data for the TfL API, and then runs the function after
  the data is loaded. BikeData is a JSON array

  d3.json("https://api.tfl.gov.uk/bikepoint").then(function(BikeData) {

    //To map the data in Mapbox, it needs to be in GeoJSON format. Going to
    create new GeoJSON

    console.log(BikeData);

    var myGeojson = {};
    myGeojson.type = 'FeatureCollection'; //Specify type is feature collection
    myGeojson.features = []; //Create features array

    var i;
    for (i = 0; i < BikeData.length; i++) { //Loop through all docking
stations

      var NbBikes = parseFloat(BikeData[i].additionalProperties[6].value);
      var NbEmptyDocks =
parseFloat(BikeData[i].additionalProperties[7].value);
      var NbTotalDocks =
parseFloat(BikeData[i].additionalProperties[8].value);
      var PropFull = NbBikes/(NbBikes+NbEmptyDocks);

      var newFeature = { //Create each docking station feature as JSON
object

        "type": "Feature",
        "geometry": {
          "type": "Point",
          "coordinates": [BikeData[i].lon, BikeData[i].lat]
        },
        "properties": {
          "NbBikes": NbBikes,
          "NbEmptyDocks": NbEmptyDocks,
          "NbTotalDocks": NbTotalDocks,
          "PropFull": PropFull
        }
      }
      myGeojson.features.push(newFeature); //Add the new feature to the
features array

    }

    console.log(myGeojson);

    map.addSource('points', { //Add the geojson to the map. Same syntax as earlier
tube station proportional circles example
      'type': 'geojson',
      'data': myGeojson
    });

    map.addLayer({
      id: 'BikeStations',

```

```

        type: 'circle',
        source: 'points',
        'layout': {
            'visibility': 'visible'
        },
        paint: {
            'circle-color': {
                property: 'PropFull',
                stops: [
                    [0, "#00f"],
                    [1, "#f00"],
                ]
            },
            'circle-opacity': 0.6,
            'circle-stroke-width': 0.1,
            'circle-stroke-color': '#999',
            'circle-stroke-opacity': 1,
            'circle-radius': {
                property: 'NbTotalDocks',
                stops: [
                    [{zoom: 9, value: 0}, 0.1],
                    [{zoom: 9, value: 40}, 2],
                    [{zoom: 12, value: 0}, 3],
                    [{zoom: 12, value: 40}, 15],
                    [{zoom: 16, value: 0}, 4],
                    [{zoom: 16, value: 40}, 60],
                ]
            }
        }
    });
});
</script>
</body>
</html>

```

There is a slightly more advanced version of this example, *API_Example2b_TfLBikeShare.html*, which adds a popup showing the detailed information on each docking station when you mouseover the circles. This example still needs a title, legend and data credit if it was going to be published online.

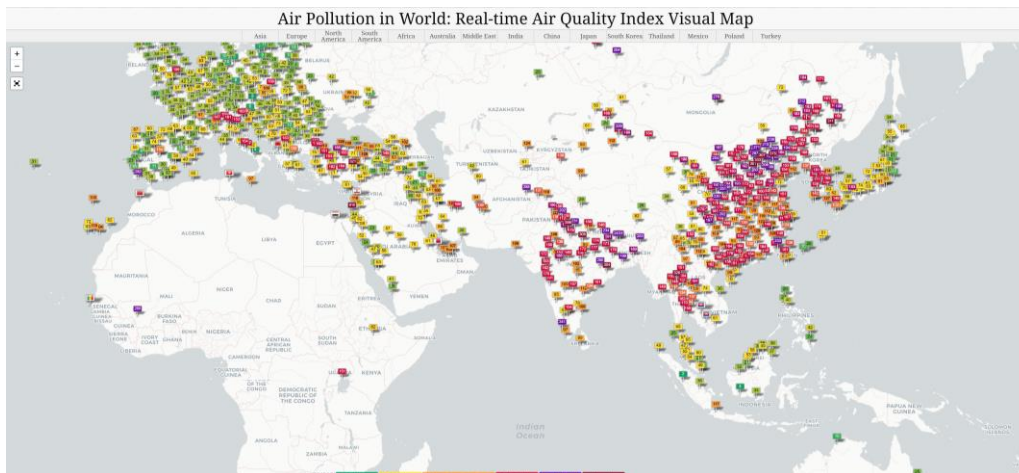
Visualising Real Time Air Quality Data

There are hundreds of APIs available online, recording many different types of live and archived data. A good place to explore lots of these is the Postman site-

<https://www.postman.com/explore>

Air quality is an increasingly important topic in city planning as the health impacts of poor air quality are becoming more widely understood. Air quality is interesting in terms of real time spatial data due to its continuous temporal variation, and diverse patterns between cities internationally. The World Air Quality Index project is an ambitious attempt to integrate the hundreds of global air quality feeds and present information using a set of consistent indicators- <http://aqicn.org/>

The website also includes an interesting API, which is described here- <https://aqicn.org/api/>



It is common for many APIs to require a token so that the number of data requests you make can be monitored. To get your own token for the AQICN data, follow the link here-

<https://aqicn.org/data-platform/token/#/>

London Real Time Air Quality Example

The first example is a table of current air quality readings for London. This is similar to the earlier tube status example-

London Real Time Air Quality Data

Data provided by [AQICN Open Data Platform](https://aqicn.org/data-platform/).

Pollutant	Status
PM2.5	37
PM10	16
O3	20.5
NO2	13.5
SO2	0.6
CO	4.6

In this example, the headings and tables have styles applied to improve the design and clarity. The main request to the AQICN API this time includes the city name and the token, so is coded using an AQuery variable as follows-

```
var city = "london";
var token = "065c9ad49f2ed519ebbd191cfc60a4b5e6dee4b7"
var AQuery = "https://api.waqi.info/feed/" + city + "?token=" + token;

d3.json(AQuery).then(function(AQData) { ... })
```


The colour of the cells showing the reading for each pollutant is set according to the value (from green for no health risk, to purple and red for health hazards). This is done using a function called SetColor in the code.

API_Example3_AirQuality.html

```
<!DOCTYPE html>
<html>
<head>
<title>London Real Time Air Quality Example</title>
<meta name="viewport" content="initial-scale=1.0, user-scalable=no" />

<script charset="utf-8" src="https://d3js.org/d3.v7.min.js"></script>

<style>
...
</style>
</head>

<body>

<div id="pagecontent">
  <h1>London Real Time Air Quality Data</h1>
  <p>Data provided by <a href="https://aqicn.org/data-platform/token/#/">AQICN Open Data
Platform</a>.</p>

  <table id="airqualitytable">
    <tr>
      <th>Pollutant</th>
      <th>Status</th>
    </tr>
    <tr>
      <td class="PName">PM2.5</td>
      <td class="PStatus" id="pm25">Data</td>
    </tr>
    <tr>
      <td class="PName">PM10</td>
      <td class="PStatus" id="pm10">Data</td>
    </tr>
    <tr>
      <td class="PName">O3</td>
      <td class="PStatus" id="o3">Data</td>
    </tr>
    <tr>
      <td class="PName">NO2</td>
      <td class="PStatus" id="no2">Data</td>
    </tr>
    <tr>
      <td class="PName">SO2</td>
      <td class="PStatus" id="so2">Data</td>
    </tr>
    <tr>
      <td class="PName">CO</td>
      <td class="PStatus" id="co">Data</td>
    </tr>
  </table>

</div>

<script>

/*The AQICN API requires a city to be specified, and the use of a token. Details here-
https://aqicn.org/api/
So we create a city variable and token variable, and combine these for the final URL*/
```

```

var city = "london";
var token = "065c9ad49f2ed519ebbd191cfc60a4b5e6dee4b7"
var AQurl = "https://api.waqi.info/feed/" + city + "?token=" + token;

d3.json(AQurl).then(function(AQData) {

    console.log(AQData);

    setColor = function(value) { // We will colour the table background according to the
indicator value. Colours from here- https://aqicn.org/json-api/demo/
        var color;
        if (value>=300){color="#7e0023";} else
        if (value>=200){color="#660099";} else
        if (value>=150){color="#cc0033";} else
        if (value>=100){color="#ff9933";} else
        if (value>=50){color="#ffde33";} else
        if (value>0){color="#009966";} else
        if (value=0){color="#cccccc";}
        return color;
    }

    var pollutants = [ // Insert pollutant values into an array
        {"name": "pm25", "value": AQData.data.iaqi.pm25.v},
        {"name": "pm10", "value": AQData.data.iaqi.pm10.v},
        {"name": "o3", "value": AQData.data.iaqi.o3.v},
        {"name": "no2", "value": AQData.data.iaqi.no2.v},
        {"name": "so2", "value": AQData.data.iaqi.so2.v},
        {"name": "co", "value": AQData.data.iaqi.co.v}
    ];

    console.log(pollutants);

    pollutants.forEach(function(pollutant) { //Loop through pollutants and enter them into the
table

        document.getElementById(pollutant.name).innerHTML = pollutant.value;
        var bgcolor = setColor(pollutant.value); //Use setColor function to get background
colour according to pollutant value
        document.getElementById(pollutant.name).style.backgroundColor = bgcolor;

    });

});

</script>
</body>
</html>

```

Real Time Air Quality for Multiple Cities

The final example, API_Example3b_AirQuality.html, allows the user to choose between a drop-down list of international cities and presents the air quality data for these-

Hombegowda Nagar, Bengaluru, India Real Time Air Quality Data

Data provided by [AQICN Open Data Platform](#).

Bangalore ▼

Pollutant	Status
PM2.5	114
PM10	67
O3	15.8
NO2	21.3
SO2	6.8
CO	16.1

The code is very similar to the previous example, with the data update process restructured as the function SetCityData.

The drop-down menu is quite a slow way for users to compare the different cities, and it would be interesting to explore different visualisation methods (maps, charts etc.) that display air quality data on multiple cities simultaneously, perhaps combining real time and archived data.

Further API Resources

We have only explored two interesting APIs in this practical. There are many hundreds available online, from aircraft positions, to social media messages, to weather updates. Try the following-

Large catalogue of APIs- <https://github.com/APIs-guru/openapi-directory>

Online service that manages APIs- <https://explore.postman.com/>

UK Government API catalogue- <https://data.gov.uk/>